Metodo Master

Materia	Algoritmi
Anno	Secondo Anno
Data	@October 17, 2024
Studiato	Done

Sia data la seguente ricorrenza:

$$T(n) = aT(\frac{n}{b}) + f(n)$$

con $a > 0, b > 1$ costanti.
Notiamo che se
$b \not| n$, avremo che $aT(\frac{n}{b}) = a'T(\lfloor \frac{n}{b} \rfloor) + a''T(\lceil \frac{n}{b} \rceil)$ per qualche costante $a', a'' > 0 : a' + a'' = a$. Possiamo sempre supporre con questa assunzione che la nostra n sia sempre divisibile per b.

Data un'equazione di tale forma, a parte distinguere a e b, possiamo distinguere due funzioni. Osservando l'albero di ricorsione, può accadere che:

- il costo della radice sia predominante rispetto ai livelli sottostanti;
- ogni livello ha lo stesso costo, quindi moltiplicare i costi di ogni livello per i numeri dei livelli;
- le foglie sono predominanti rispetto ai livelli soprastanti. Anche se il costo di ogni foglia è $\Theta(1)$, potrebbe accadere che il numero di foglie è nettamente superiore rispetto agli altri nodi.

Il metodo master permette di distinguere questi casi in base alla funzione $f(n)$, in base alla divisione del problema e in base al numero di foglie. Definiamo allora due funzioni:

- **driving function**, ossia la **funzione guidante**, $f(n)$;

- **watershed function**, ossia la **funzione spartiacque**, ossia la funzione $w(n) = n^{log_b a}$

Allora vale il **teorema master**.

Teorema master

> *Siano $a > 0, b > 1$ due costanti. Sia $f(n)$ una funzione definita e non negativa su tutti i numeri reali sufficientemente grandi. Sia:*
>
> $$T(n) = aT(\frac{n}{b}) + f(n)$$
>
> *Allora:*
>
> 1. $\exists \epsilon > 0 : f(n) = O(n^{\log_b a - \epsilon}) \implies T(n) = \Theta(n^{\log_b a})$;
> 2. $\exists k \geq 0 : f(n) = \Theta(n^{\log_b a} \cdot \log^k n) \implies T(n) = \Theta(n^{\log_b a} \cdot \log^{k+1} n)$;
> 3. $\exists \epsilon > 0 : f(n) = \Omega(n^{\log_b a + \epsilon})$ *e* $f(n)$ *soddisfa la condizione di regolarità, ossia:*
>
> $$\exists c < 1 : af(\frac{n}{b}) \leq cf(n)$$
>
>   *si ha che $T(n) = \Theta(f(n))$.*

Il caso 1 significa che la driving function viene dominata dalla watershed function. Ma anche ponendo un possibile ϵ piccolo, la watershed function continua a dominare. Quindi $w(n)$ cresce asintoticamente più velocemente rispetto a $f(n)$; più precisamente, possiamo dire che $w(n)$ e $f(n)$ differiscono per un polinomio $\Theta(n^\epsilon)$. Quindi significa che nell'albero di ricorsione, il costo dei livelli cresce in

maniera **almeno geometrica** dalla radice alle foglie e il costo delle foglie domina il costo totale dei rami interni.

Il caso 2 indica che ogni livello ha orientativamente lo stesso costo. Quindi $w(n)$ cresce asintoticamente in maniera simile a $f(n)$ ed esse differiscono per un fattore polilogaritmico $\Theta(\log^k n)$. Tutti i livelli dell'albero di ricorsione quindi hanno approssimativamente lo stesso costo $\Theta(n^{\log_b a} \cdot \log^k n)$ e ci sono $\Theta(\log n)$ livelli.

Nota: in realtà l'altezza dell'albero di ricorsione è $\log_b n$, pertanto $\Theta(\log n)$ indica l'ordine di grandezza dell'albero e non il numero effettivo di livelli. Nonstante ciò, anche risolvendo utilizzando la tecnica dell'albero di ricorsione, si dimostra che la complessità in questi casi contiene un fattore logaritmico in più $\log n$.

Infine, nel caso 3 $w(n)$ cresce in maniera asintoticamente più lenta di $f(n)$ ed esse differiscono per un fattore polinomiale $\Theta(n^\epsilon)$. Quindi il costo dei livelli decresce **almeno geometricamente** e il costo della radice domina quello dei nodi.

Esempio

Sia data la ricorrenza:

$$T(n) = 4T(\frac{n}{2}) + n^{1.99}$$

Si ha che $f(n) = n^{1.99}, \omega(n) = n^{log_2 4 = 2}$. Notiamo facilmente che $f(n) = O(n^{2-\epsilon})$, considerando $0 < \epsilon \leq 0.01$. Quindi possiamo dire che $T(n) = \Theta(n^2)$.

Sia invece data:

$$T(n) = 3T(\frac{n}{4}) + n \log n$$

Si ha $f(n) = n \log n$ e $\omega(n) = n^{\log_4 3} = n^{0,793}$. Allora siamo nel caso in cui $f(n) = \Omega(n^{\log_4 3 + \epsilon})$. Allora il valore di epsilon dev'essere compreso tra 0 e $1 - 0,793 = 0,207$.

Bisogna soddisfare la condizione di regolarità, ossia:

$$\exists c < 1 : af(\frac{n}{b}) \leq cf(n)$$

Ossia dire che:

$$3\frac{n}{4} \cdot \log \frac{n}{4} \leq cn \log n$$

E risolvendo si ha:

$$\frac{3}{4}n(\log n - \log 4) \leq cn \log n \implies \frac{3}{4}n \log n - \frac{3}{2}n \leq cn \log n$$
$$\implies n \log n(\frac{3}{4} - c) \leq \frac{3}{2}n$$

Il termine a sinistra sarà più piccolo solamente se $\frac{3}{4} - c$ è una quantità minore o uguale a zero, e questo si ha per:

$$\frac{3}{4} \leq c < 1$$

E quindi $T(n) = \Theta(n \log n)$.

Infine, vediamo il merge sort:

$$T(n) = 2T(\frac{n}{2}) + \Theta(n)$$

Sappiamo $f(n) = \Theta(n), \omega(n) = n^{\log_2 2} = n$. Allora esiste $k \geq 0$ tale che $f(n) = \Theta(n \cdot \log^k n)$. Infatti basta porre $k = 0$. Si ha quindi che $T(n) = \Theta(n \log^1 n) = \Theta(n \log n)$.

Quando applicare il teorema master

Ci sono situazioni in cui **non è possibile applicare il teorema master** nonostante l'equazione sia del tipo master. In particolare è possibile che:

- ci sia un **gap** tra il caso 1 e il caso 2;
- ci sia un **gap** tra il caso 2 e il caso 3;
- $f(n)$ e $\omega(n)$ non sono **asintoticamente comparabili**;
- $w(n)$ è **dominato da** $f(n)$ **ma** $f(n)$ **non è regolare**.

Ad esempio, sia:

$$T(n) = 2T(\frac{n}{2}) + \frac{n}{\log n}$$

Avremo $a = b = 2, f(n) = \frac{n}{\log n}, \omega(n) = n^{\log_2 2} = n$. Sicuramente possiamo dire che non siamo nel caso 3, in quanto $f(n)$ cresce più lentamente di $w(n)$.
Inoltre,
$\forall \epsilon \geq 0, \log n = o(n^\epsilon) \implies \frac{1}{\log n} = \Omega(n^{-\epsilon}) \implies \frac{n}{\log n} = \Omega(n^{1-\epsilon}) \implies \frac{n}{\log n} \neq O(n^{1-\epsilon})$. Quindi non siamo nel caso 1.

Non siamo neanche nel caso 2 perché $\frac{n}{\log n} = \Theta(n \log^k n) \iff k = -1$, ma dev'essere $k \geq 0$, quindi non è possibile.

Esempio: Heapify

Il tempo necessario per correggere la relazione tra $A[i], A[i.left], A[i.right]$ è $\Theta(1)$. L'albero di ricorsione, inoltre, potrebbe non essere completo. Possiamo dire che nel caso peggiore:

$$T(n) \leq T(\frac{2}{3}n) + \Theta(1)$$

n è la dimensione considerata, ossia il numero di nodi. L'altezza del nodo a cui i sottoalberi sono radicati è $\log n$.

Poiché è una disuguaglianza, il risultato è O e non Θ.

Sappiamo che $a = 1, b = \frac{3}{2}, f(n) = \Theta(1), \omega(n) = n^{\log_{\frac{3}{2}} 1} = n^0 = 1$

$f(n) = \Theta(1 \log^0 n) \implies T(n) = O(\log n)$.

Quindi eseguire `heapify` su un sottoalbero con altezza h impiega $O(h)$.

Analisi di build-max-heap

Ricordiamo che lo pseudo-codice della procedura è il seguente:

```
build-max-heap(A, n):
   for i = floor(n/2) to 1 do:
      max-Heapify(A,i)
```

Tale procedura ha complessità $O(n \log n)$. Generalmente `heapify` applicato ad un sottoalbero con altezza h impiega $O(h)$. Quindi possiamo dire che:

$$T(n) \leq \sum_{h=0}^{\lfloor \log n \rfloor} ch\lceil \frac{n}{2^{h+1}} \rceil \leq ch\frac{n}{2^h} = cn \sum_{h=0}^{\lfloor \log n \rfloor} \frac{h}{2^h} \leq cn \sum_{h=0}^{\infty} \frac{h}{2^h} =$$
$$= \sum_{h=0}^{\infty} h(\frac{1}{2})^h = cn \frac{\frac{1}{2}}{(1-\frac{1}{2})^2}$$

Dove $\frac{n}{2^{h+1}}$ è uguale al numero di nodi di altezza h e $\lceil x \rceil \leq 2x, |x| \geq \frac{1}{2} \implies \lceil \frac{n}{2^{h+1}} \rceil < \frac{n}{2^h}$.